

AI 기반 회로 문제 풀이 지원 시스템

전자공학기초실험
김규한, 남채현, 이승아

목차



01

주제 선정 이유

- 회로이론 학습의 어려움
- 전류 흐름의 직관적 이해 부족
- 실제 동작과 계산 비교의 어려움

02

프로젝트 목적

- 전자정보공학부 학생 대상
- 회로 문제 풀이 지원
- 최소 목표 기반 코드 구성

03

프로그램 핵심 기능

- 키르히호프 법칙 적용
- 옴의 법칙 적용
- 회로 동작 시각화

04

사용한 AI 도구

- Claude.ai, Cursor, Gemini
- API 키 결제 사용
- AI 기반 회로 인식 기술

05

웹사이트 구조 및 작동 방식

- 회로 사진 업로드 기능
- AI 기반 회로 인식
- 자동 풀이 및 해답 제공

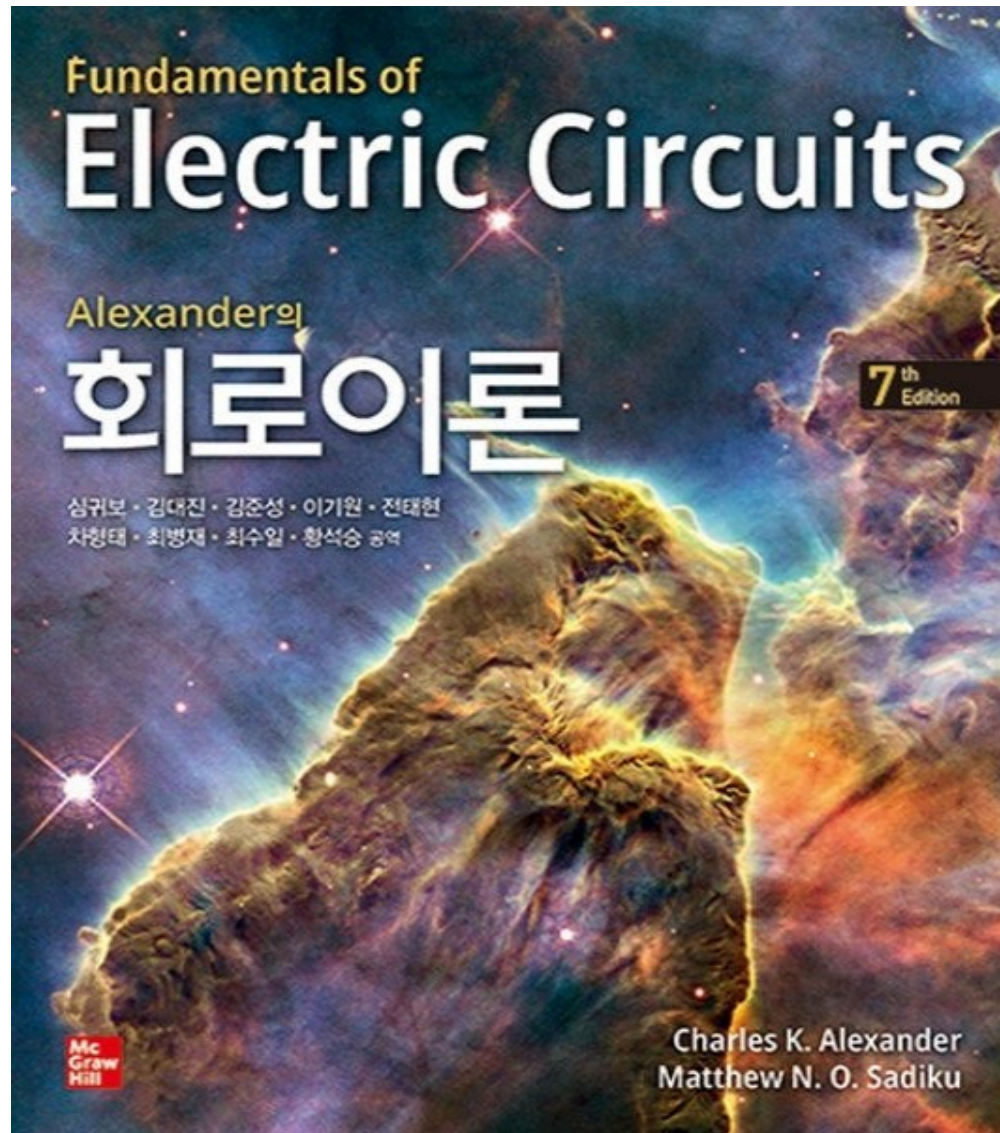
06

회로 코드 설명 및 데모

- 코드 구조 및 알고리즘
- 데모 영상 시연
- 프로젝트 성과 및 질의응답

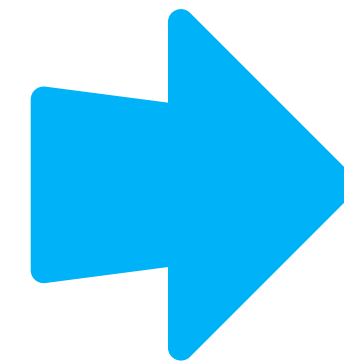


주제 선정 이유



회로이론 학습의 어려움

- 전자정보공학부 학생들이 회로이론의 복잡한 수식과 계산으로 어려움을 겪음
- 챗gpt등 AI가 회로 인식을 정확히 하지 못함



회로해석 전용
AI 문제풀이를
지원하는 앱
을 만들면 어떨
까?

프로젝트 목적

2.27 그림 2.91의 회로에서 전류 I_o 를 계산하라.

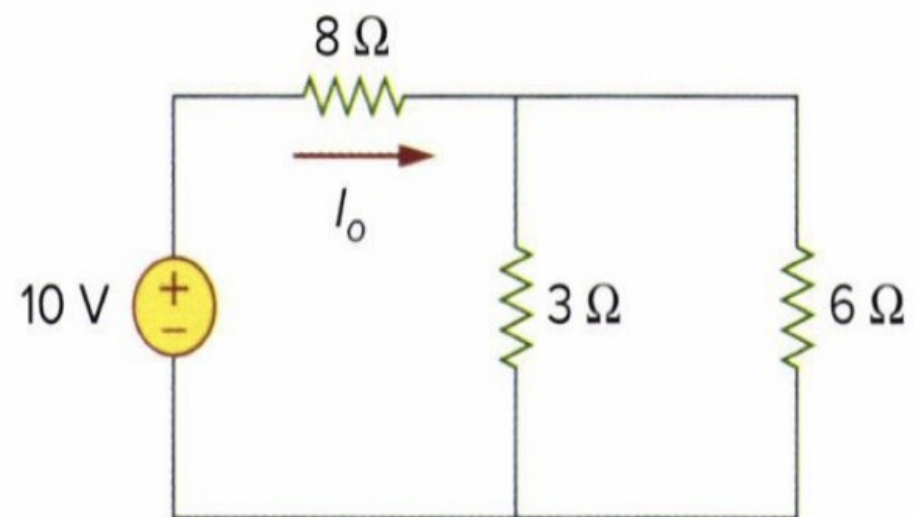
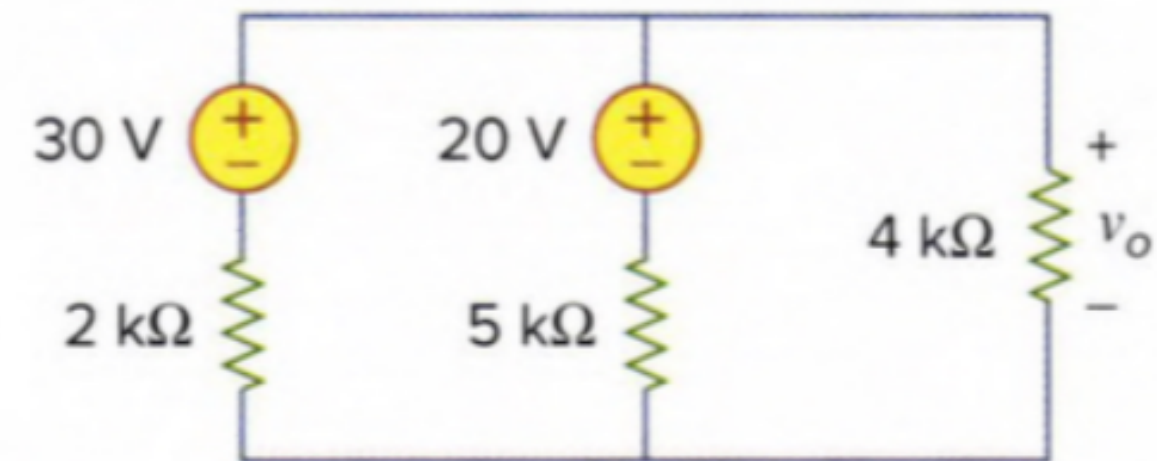


그림 2.91
문제 2.27.

5 그림 3.54의 회로에서 v_o 를 구하라.



프로그램 주요 기능

키르히호프 법칙 적용

회로의 각 절점에서 전류의 합이 0이 되는 KCL과 폐회로의 전압 합이 0이 되는 KVL을 자동으로 적용하여 회로 해석

옴의 법칙 적용

저항, 전압, 전류의 관계식 $V=IR$ 을 활용하여 회로 내 각 소자의 전압과 전류 값을 자동으로 계산

회로 동작 시각화

계산된 전류의 흐름을 시각적으로 표현하여 사용자가 회로의 실제 동작을 직관적으로 이해할 수 있도록 지원



사용한 AI 도구

Claude.ai

- Anthropic의 대화형 AI 모델
- 코드 생성 및 회로 해석 로직 개발에 활용
- 복잡한 알고리즘 구현 시 자문 역할
- 자연어 처리 기능을 통한 사용자 인터페이스 개선

Cursor

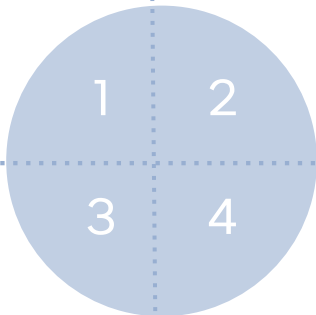
- AI 기반 코드 에디터
- 실시간 코드 자동완성 및 오류 수정
- 개발 생산성 향상에 기여
- 회로 분석 코드 작성 시 효율적인 개발 환경 제공

Gemini

- Google의 멀티모달 AI 모델
- 회로 이미지 인식 및 분석에 활용
- 업로드된 회로 사진에서 소자와 연결 구조 파악
- OCR 기술을 통한 회로 기호 인식

유료 API 키 결제 사용

- 실제 AI 서비스 이용을 위해 API 키 결제
- Gemini API를 통한 이미지 분석 기능 구현
- 안정적인 서비스 제공을 위한 유료 플랜 사용
- 프로젝트의 핵심 기능 구현에 필수적



웹사이트 구조 및 작동 방식

AI 기반 회로 해석 웹 플랫폼

사용자가 회로 사진을 웹사이트에 업로드하면, AI가 이미지 속 회로를 자동으로 인식하고 분석

→ 키르히호프 법칙과 옴의 법칙을 적용하여 회로의 전류와 전압을 계산하고, 최종 해답과 상세한 풀이 과정을 제공

회로 사진 업로드

- 이미지 파일 선택
- 웹 인터페이스 업로드
- 회로 요소 자동 추출
- OCR 기술 활용

AI 자동 인식 및 풀이

- Claude.ai API 사용
- 회로 구조 파싱
- 자동 계산 수행
- 결과 및 풀이 출력



회로 코드 설명 (I)

```
eline (1).py

ings Help

Sonnet Vision API 기반 이미지 속해 및 자동 명물찾 분기 파이프라인

xn - 이미지 속해 & 구조화 JSON 추출
_question_text (문항 번호 제거, ** 마크업 없음)
n_type          CONVERSION | CALCULATION
lutions[]       메뉴 문거별 initial_circuit + 풀이 경보

:라인 분기
ATION → run_full_pipeline(initial_circuit)

:조립

_type, parsed_question_text,
rtions: [
_id, menu_title, initial_circuit,
lt_circuit, solution, steps, applied_theories ]

r2.py - run_full_pipeline(), , build_steps(), build_applied_theories()
- pip install anthropic
```

AI

- AI기반 코드 해석

```
C:\> Users\ch936\OneDrive\문서\page.tsx > ...

429 const Page2 = ({ isSuccess, onDone }: { isSuccess: boolean; onDone: () => void }) => {
442
443   return (
444     <div style={
445       {
446         minHeight: "100vh",
447         display: "flex",
448         flexDirection: "column",
449         alignItems: "center",
450         justifyContent: "center",
451         background: "var(--bg)",
452         paddingTop: "56px",
453         animation: "pageIn 0.5s ease",
454       }
455     >
456       <div style={{ animation: "fadeSlideUp 0.6s ease", textAlign: "center" }}>
457         <ResistorAnimated />
458         <p style={{ marginTop: "24px", fontFamily: "'Noto Sans KR', sans-serif", fontS
459           {loadingText}
460         </p>
461       </div>
462     </div>
463   );
464 };
465
466 const AccordionItem = ({ step, idx }: { step: AccordionStep; idx: number }) => {
467   const [open, setOpen] = useState(idx === 0);
468   return (
469     <div style={
470       {
471         border: "1px solid var(--border)",
472         borderRadius: "10px",
473         overflow: "hidden",
474       }
475     >
```

page.tsx(프르트엔트)

- 웹 디자인 구성
- 풀이를 단계별로 출력

```
Jupyter main.py Last Checkpoint: 4 hours ago

File Edit View Settings Help

1 |"""
2 |main.py
3 |
4 |회로 해석 웹 서비스 - FastAPI 서버 진입점 (v3 - 멀티모달 Vision 통합)
5 |
6 |엔드포인트 구성
7 |
8 |GET / 헬스체크
9 |POST /api/solve 텍스트 회로 JSON → 수학 해석 (4단계 기존 기능 유지)
10 |POST /api/upload-image 회로 이미지 파일 → Vision 분석 → 전체 파이프라인
11 |
12 |필수 환경변수
13 |
14 |ANTHROPIC_API_KEY Claude API 키
15 |(선택) ALLOWED_ORIGINS 웹표 구분 도메인 목록 (기본값: *)
16 |
17 |실행 방법
18 |
19 |pip install fastapi uvicorn anthropic sympy python-multipart
20 |python main.py
21 |→ http://localhost:8000/docs (Swagger UI)
22 |"""
23 |
24 |from __future__ import annotations
25 |
26 |import asyncio
27 |import os
28 |import traceback
29 |from contextlib import asynccontextmanager
30 |from functools import partial
31 |from typing import Any, List, Optional
32 |
33 |import anthropic
```

main

- 두 코드를 호환



회로 코드 설명 (2)

계산 로직

키르히호프 법칙과 옴의 법칙을 적용하여 연립방정식을 구성하고, 행렬 연산을 통해 각 노드의 전압과 브랜치의 전류를 계산(관련된 식을 제공)

출력 형식

계산된 전류 및 전압 값을 사용자 친화적인 형식으로 표시하며, 단계별 풀이 과정과 최종 해답을 명확하게 제공

오류 처리

회로 인식 실패, 불완전한 회로, 계산 불가능한 구조 등의 예외 상황을 감지하고 적절한 오류 메시지를 반환

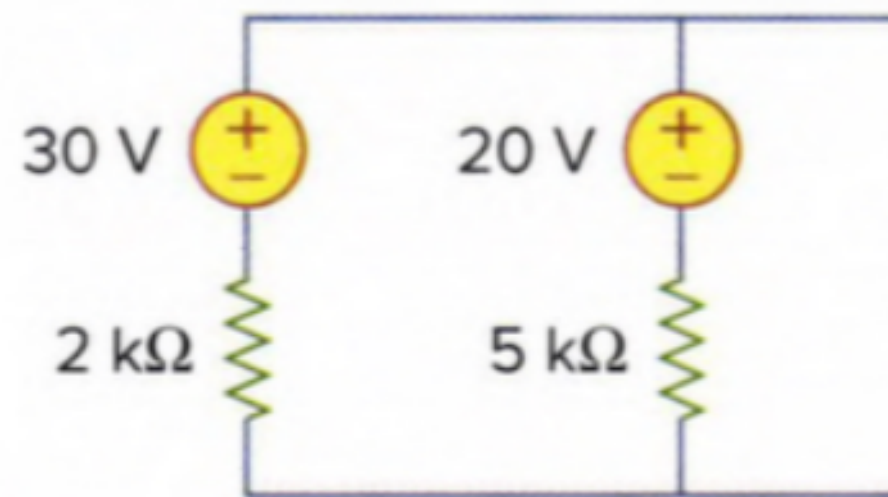


데모 영상



결론

문 3.54의 회로에서 v_o 를 구하라



목표 달성

- 최소 목표 문제를 달성함



느낀점(어려웠던점)

- 자연어 처리 및 멀티모달 인식 오류
- 클라이언트&서버 간 데이터 정합성 문제



향후 개선방안

- 더 복잡한 회로 지원
- 인식 정확도 향상
- 등가회로의 시각화 출력기능



질의응답

궁금하신 점을 자유롭게 질문해 주세요
깃허브 링크: <https://github.com/kkeeham/circuitsolver>

감사합니다